

162. Find Peak Element

Difficulty: Medium

Patterns: Array, Binary Search, Slope Search

Source: <https://leetcode.com/problems/find-peak-element/description/>

1. Problem Summary

We are given a 0-indexed integer array `nums`.

A peak is an index `i` whose value is strictly greater than its neighbors. The problem lets us imagine two virtual boundary values:

```
nums[-1] = -infinity
nums[n] = -infinity
```

So the first element only needs to be greater than `nums[1]`, and the last element only needs to be greater than `nums[n - 2]`.

Return the index of any peak. If multiple peaks exist, any valid peak index is accepted.

2. Constraints and Observations

- `1 <= nums.length <= 1000`
- `-231 <= nums[i] <= 231 - 1`
- `nums[i] != nums[i + 1]` for every valid adjacent pair.
- The required runtime is $O(\log n)$.
- The array is not sorted, so this is not normal binary search on values.
- Adjacent values are never equal, so every adjacent step is either strictly upward or strictly downward.
- At least one peak always exists because the outside values are treated as `-infinity`.

3. Pattern Recognition

This is a binary search problem, but not because the array is sorted.

The searchable structure is the local slope:

- If `nums[mid] < nums[mid + 1]`, we are moving upward from `mid` to `mid + 1`. A peak must exist somewhere to the right.
- If `nums[mid] > nums[mid + 1]`, we are moving downward from `mid` to `mid + 1`. A peak must exist at `mid` or somewhere to the left.

This is often called binary search on slope or binary search on answer direction.

4. Naive Approach

Scan every index and check whether it is greater than both neighbors.

To avoid boundary special cases, treat missing neighbors as `-infinity`.

This works, but it takes $O(n)$ time. The problem explicitly asks for $O(\log n)$, so we need to discard about half the search space each step.

5. Key Intuition

The key question is not "is `nums[mid]` the peak?"

The better question is:

Which side is guaranteed to contain a peak?

Look only at `nums[mid]` and `nums[mid + 1]`:

- If the slope goes up, keep the right half.
- If the slope goes down, keep the left half.

Why is that enough?

If we move in the direction of a larger neighbor, the values cannot keep increasing forever because the outside boundary is `-infinity`. Eventually, the sequence must either turn downward, creating a peak, or reach the boundary element, which is itself a peak.

6. Optimal Solutions Ranked

Variant 1: Linear Scan

- Idea: Return the first index `i` where `nums[i] > nums[i + 1]`; if none exists, return `n - 1`.
- Time: $O(n)$
- Space: $O(1)$
- Tradeoff: Very simple and valid logically, but it fails the required $O(\log n)$ runtime.

Variant 2: Binary Search With Explicit Peak Check

- Idea: At each `mid`, compare both neighbors. If `mid` is not a peak, move toward the greater neighbor.
- Time: $O(\log n)$
- Space: $O(1)$
- Tradeoff: Correct, but it has more boundary handling and more comparisons than necessary.

Variant 3: Binary Search on Slope

- Idea: Compare only `nums[mid]` and `nums[mid + 1]`. Move toward the side that must contain a peak.
- Time: $O(\log n)$
- Space: $O(1)$
- Tradeoff: This is the cleanest optimal form. It avoids checking both neighbors and handles boundaries naturally by searching while `left < right`.

No solution can asymptotically beat $O(\log n)$ under the required comparison-style search expectation. We can do $O(1)$ only for special input shapes, not for every valid array.

7. Most Optimal Approach

Use two pointers, `left = 0` and `right = n - 1`.

While more than one index remains:

1. Let `mid = (left + right) // 2`.
2. Compare `nums[mid]` with `nums[mid + 1]`.
3. If `nums[mid] > nums[mid + 1]`, the slope is falling, so a peak exists in `[left, mid]`. Set `right = mid`.

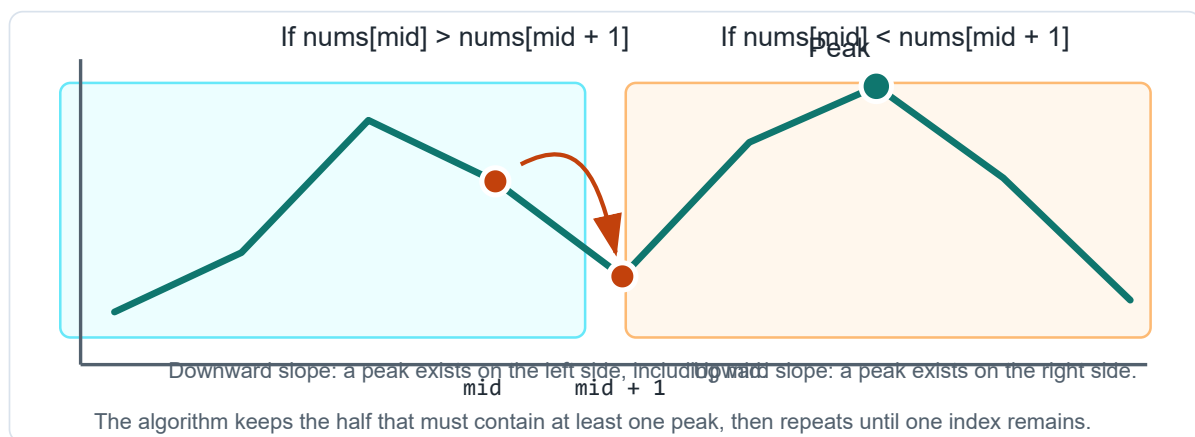
4. Otherwise, the slope is rising, so a peak exists in `[mid + 1, right]`. Set `left = mid + 1`.

When `left == right`, the remaining index is a peak.

8. Algorithm

1. Initialize `left = 0` and `right = len(nums) - 1`.
2. While `left < right`:
 - Compute `mid = (left + right) // 2`.
 - If `nums[mid] > nums[mid + 1]`, set `right = mid`.
 - Otherwise, set `left = mid + 1`.
3. Return `left`.

9. Visual Explanation



Comparing adjacent values tells us which half must contain at least one peak.

The important detail is that we never throw away all possible peaks. Each decision keeps a range that is guaranteed to contain at least one peak.

10. Dry Run

Input:

```
nums = [1, 2, 1, 3, 5, 6, 4]
```

Expected: index `1` or index `5` is valid. The binary search below returns `5`.

Step	left	right	mid	nums[mid]	nums[mid + 1]	Decision
1	0	6	3	3	5	Rising, so peak is to the right. <code>left = 4</code>
2	4	6	5	6	4	Falling, so peak is at mid or left. <code>right = 5</code>
3	4	5	4	5	6	Rising, so peak is to the right. <code>left = 5</code>

Now `left == right == 5`, so return 5.

Check:

```
nums[5] = 6
nums[4] = 5
nums[6] = 4
```

Since `6 > 5` and `6 > 4`, index 5 is a valid peak.

11. Correctness Argument

Invariant: At the start of every loop iteration, the search interval `[left, right]` contains at least one peak.

Initially, `[0, n - 1]` contains at least one peak because the virtual values outside the array are `-infinity` and adjacent elements are unequal.

During an iteration:

- If `nums[mid] < nums[mid + 1]`, the slope rises from `mid` to `mid + 1`. Starting at `mid + 1` and moving right, either the array eventually falls, creating a peak, or it reaches index `right`, which is still higher than the virtual right boundary if the rise never stops. Therefore, `[mid + 1, right]` contains a peak.
- If `nums[mid] > nums[mid + 1]`, the slope falls from `mid` to `mid + 1`. Moving left from `mid`, either the array eventually rises into `mid`, making `mid` or a left-side turning point a peak, or it reaches index `left`, which is higher than the virtual left boundary if the fall never stops. Therefore, `[left, mid]` contains a peak.

So every update preserves the invariant.

The loop stops only when `left == right`. At that point, the interval contains exactly one index and, by the invariant, that index must be a peak. The algorithm returns that index, so it returns a valid answer.

12. Complexity Analysis

- Time: $O(\log n)$ because each iteration keeps roughly half of the remaining interval.
- Space: $O(1)$ because the algorithm uses only a few variables.

13. Edge Cases

- Single element: return `0`, because it is greater than both virtual boundaries.
- Strictly increasing array: return `n - 1`.
- Strictly decreasing array: return `0`.
- Multiple peaks: return any one of them.
- Peak near an edge: the virtual `-infinity` boundary makes edge peaks valid.
- Negative values: comparisons still work exactly the same way.

14. Final Clean Code

```
from typing import List

class Solution:
    def findPeakElement(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1

        while left < right:
            mid = (left + right) // 2

            if nums[mid] > nums[mid + 1]:
                right = mid
            else:
                left = mid + 1

        return left
```

15. Key Takeaways

- The array is not sorted, but the local slope gives binary-search direction.

- Comparing `nums[mid]` and `nums[mid + 1]` is enough.
- Move toward the larger neighbor because that side must contain a peak.
- The invariant is: the current interval always contains at least one peak.
- The final one-index interval is therefore guaranteed to be a valid peak.